

Stem Agent - Project Report

APPROACH

The core question this project tries to answer is: can an AI agent figure out what it needs to become, rather than being told? Most agent frameworks today require a human to design the architecture upfront; this includes defining prompts, wiring the tools, and deciding the flow. The concept of stem agent proposes a difference vision. You give it a task domain, and it designs itself. This idea comes from biology, where a stem cell starts as nothing specific, it reads signals from its environment and decides how much it must learn to answer the environment. If you apply that to AI agents, you get a system that starts small and grows into a specialist.

I designed the implementation around three components:

- The **builder** is an LLM that proposes new capabilities one at a time.
- The **evaluator** is a separate LLM that checks each proposal against quality rubric and either approves or rejects it with a reason.
- The **traverser** runs the final specialized agent at inference time by walking the graph in topological order and passing each node's output into the next.

Everything the agent learns about itself is stored as a directed knowledge graph, in which nodes are capabilities, edges are relationships between them. The Stem Agent is first born, in this birth phase, the agent searches the web for information about its target domain, it then synthesizes a domain knowledge node from the results and adds it to the graph. This is the agent reading "signals from its environment". I interpreted this stage as the only moment an agent knows absolutely nothing and looks at what needs to be done to evolve. The next step for the agent is to grow, this allows the agent to expand, learn and specialize. Growth happens in cycles; in each cycle, the builder looks at the current graph and proposes one new node to add. A node is either a `prompt` (an instruction for an LLM to follow at inference time) or a `tool` (a Python function the agent can call). The evaluator checks the proposal and rejects it if the content is too vague, not actionable, or structurally wrong. Rejected nodes are removed and the builder retrieves the feedback. This continues until the evaluator confirms the graph is fully specialized, or until the cycle limit is reached.

Architecture Justification

The architecture I planned and designed is the result after a couple of research papers I have read to know more about the topic of self-evolving agents.

The biological pluripotency metaphor, an undifferentiated agent core that specializes socioenvironmental signals, is inspired by recent work on STEM agents (Shen and Shen, 2026), which introduces an architecture built around an "undifferentiated agent core" that specializes into

protocol handlers, tool bindings, and memory subsystems using biology-inspired mechanisms for skill acquisition. The paper demonstrates that this approach can consolidate multiple rigid interaction protocols behind a single adaptive interface. The stem agent extends this metaphor in a different direction: *rather than specializing in communication protocols, it specializes into domain-specific reasoning pipelines, growing its own architecture node by node*. The two-node genesis I came up with (builder and evaluator) follows the same principle. My choice of implementing knowledge graph architecture as the agent's self-model was motivated by AriGraph (Anokhin et al., 2024), which shows that structured graph-based representations outperform unstructured memory approaches, this including full-history context and retrieval-augmented generation, for tasks that require reasoning and planning. AriGraph applies this to an agent's memory of the external world; in my project the same idea is extended to the agent's own architecture. I thought that rather than storing what happened, the graph stores what the agent has become: its capabilities, strategies, and tools. I thought that this would make the stem structure inspectable and easy to grow rather than implicit. The steps of the growth loop are *proposed, evaluate, commit, iterate*, I have been inspired after reading Gödel Agent (Yin et al., 2024). This paper demonstrated that LLM-driven recursive self-improvement can outperform hand-crafted agents when guided only by high-level objectives. Where Gödel Agent rewrites its own code, the stem agent grows its self-model node by node under the supervision of a separate evaluator that acts as a structural immune system, preventing the instability that pure self-modification can cause. The result is an architecture that combines biological inspiration, structured memory, and recursive growth into a single coherent system.

Experiments

I tested the stem agent on three domains: *deep research*, *python analyzer*, and *log analyst*.

The **deep research** agent was the first one I got working correctly. It built a clean four-node pipeline: a domain knowledge node that frames the question, a strategy node that breaks it into sub-queries, a `web_search` tool call, and a final strategy node that synthesizes the results into a readable answer. I asked the question: "What are the latest advancements in quantum computing?", the output was well-structured, it reported answers and real web links connected to each. This was the first sign that the pipeline pattern was working correctly.

The **python analyzer** agent was the most interesting because it created a new tool during growth called `extract_python_code`, this parses code structure using regex. I asked the agent to review a function with a bug (cylinder volume func. was computing $\pi * h^2 * r$ instead of $\pi * r^2 * h$), the agent caught the formula error, explained it correctly and showed the correct code in correct markdown code block.

The **log analyst** agent also created a custom tool (`parse_logs`) that extracts structured records from raw log lines. This confirmed to me that the sandboxed dynamic tool creation was working as intended. Even this, the answer was not perfect, but I was happy to see that the agent learned a new skill that used to solve the problem.

What Surprised Me

The most surprising thing was how consistently the builder learned to create tools without being told which tool to create for a specific domain. Given only the task class and the graph state, it would propose Python functions that made sense for the domain. Of course, I implemented an evaluation rule that dictates the rules to follow for the agent to grow. Nevertheless, I expected to have to guide this more explicitly, but it was not needed.

I was also surprised by how fragile the prompt structure for strategy nodes turned out to be. Initially, the builder kept writing strategy node content that described the domain rather than instructing an LLM what to do with an input. I was surprised that the stem agent seemed not to understand the distinction between "here is how deep research works" and "given the research question, identify three sub-topics and write a search query for each". Another interesting remark is that the agent seems to determine correctly whether a node is useful at inference time. It took several iterations of prompt engineering on the builder's system prompt to get this right consistently.

I was very surprised at how well the relation between the evaluator and builder agent turned out to be. The evaluation rules dictated at the beginning help the builder not to hallucinate. Also, every time the evaluator rejected a node, I was pleased to see that the feedback was useful for the builder to make no mistakes. It felt like the evaluator and builder are somehow more connected, because usually only after one feedback, the builder would make no mistakes.

What Failed

Several things broke during development, and some issues required multiple attempts to fix properly. I can summarize them in the list below:

- In the beginning, the builder kept adding tool nodes as the last node in the graph, which meant the tool's output was never interpreted, that just printed raw JSON to the user. A lot of prompt engineering was needed to fix this problem. In the builder prompt I added a "pipeline completeness rule", this teaches the LLM that tool nodes must always be followed by a strategy node that reads their output.
- Downstream strategy nodes were not seeing the original user input; they only saw the output of the previous node. This meant a strategy node early in the pipeline would process the domain knowledge analysis, but the tool that followed it would receive a more normalized schema

instead of the raw data it needed. The fix was to always append the original task input to every node's primary input, so every node in the chain has access to what the user originally asked.

- The first version of the evaluator was causing a lot of problems. Initially, the evaluator was very strict, it checked every created node, and it rejected every time. The result was a poor and small graph without any feature or strategy. In addition to this, the evaluation rubric only checked for basic structural validity, like does the node have a valid type and role, or is it connected. This let through many nodes with vague or descriptive content that looked valid but produced useless output at inference time. I ended up changing the entire structure and logic of the evaluator LLM, I removed the immediate evaluator check after the creation of each node, instead, now I have a function that calls the evaluator with a less rigid prompt, also the evaluator is the one who decides when to save a new checkpoint or stop the builder. The prompter manages to keep track, because it has full view over the entire graph every time.

What I Would Do With More Time

Continuous web search during growth. Right now the agent reads the web only once, at birth. During the growth loop it only looks at the internal graph state and evaluator feedback. A stronger version would periodically search for new information about the domain during growth — especially when the evaluator rejects a node, it could search for examples of what that kind of node should do.

Performance-based specialization check. The current `is_fully_specialized` asks whether the graph has the right kinds of nodes. A better check would run the agent on a few sample inputs for the domain and evaluate the quality of the outputs before declaring it ready. This is an important thing to add, since structural correctness and functional quality are not the same thing.

Post-execution feedback loop. Once a stem agent is running in production, there is currently no way for its real-world performance to influence the graph. A more real stem of behavior would be auto-refinement mode: if the agent produces a bad answer, it triggers a targeted growth cycle that adds or replaces the node most likely responsible. This also can extend to self-learning during inference, for example, the agent can recognize the task is in the domain, but it has no idea how to solve it, so it can "ask a hint" from the web tool.

Parallelism for growth and inference. The current growth loop is fully sequential, the flow goes: one node is proposed, evaluated, and committed before the next cycle starts. In practice the builder and evaluator make independent LLM calls that could run in parallel across multiple candidate nodes, with the best one selected. Similarly, at inference time, strategy nodes that don't depend on each other could be executed in parallel rather than waiting in a strict topological order. Both changes would significantly reduce the time it takes to both grow and run a specialized agent.

References

- [1] Shen, Y., & Shen, H. (2026). *_STEM Agents: Modular Self-Adapting Architectures for Dynamic Task Specialization_*. arXiv:2603.22359
- [2] Anokhin, P., et al. (2024). *_AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents_*. arXiv:2407.04363
- [3] Yin, X., et al. (2024). *_Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement_*. arXiv:2410.04444